

Notes de présentation — Soutenance CDA zØmbie zØne

Stéphane Rochard · 9 juin 2026 · 40 min présentation + 45 min entretien technique

SLIDE 01 — Projet & Candidat

Ce que tu dis :

"zØmbie zØne, c'est un site de réservation pour un parc fictif post-apocalyptique. L'idée c'est de couvrir tout le spectre d'une application métier réelle : une vitrine publique immersive, un espace client pour réserver des sessions d'activités, et un backoffice d'administration complet.

Le projet est réalisé en solo, ce qui m'a contraint à endosser tous les rôles : Product Owner pour définir le backlog, Scrum Master pour gérer les sprints, architecte pour les choix techniques, et DevOps pour le déploiement. C'est un contexte professionnel simulé, mais rigoureux — CI verte obligatoire avant chaque merge, commits conventionnels, branches par feature."

Points à valoriser : solo = maîtrise bout en bout. 4 acteurs = modèle RBAC réel.

SLIDE 02 — Cahier des charges Visiteur & Membre

Ce que tu dis :

"J'ai défini deux profils utilisateur distincts. Le visiteur peut parcourir le catalogue, filtrer par catégorie, voir les sessions disponibles et s'inscrire. Le membre connecté accède à tout ça, plus la gestion du panier, le passage de commande, l'historique, et la gestion de son compte — y compris la suppression soft delete pour respecter le RGPD.

La cible, c'est les 16-30 ans, mobile-first, design horror. Ça a dicté des choix concrets : skeleton loading, pagination, lazy loading, images WebP."

SLIDE 03 — Cahier des charges Admin & Contraintes

Ce que tu dis :

"L'admin dispose d'un backoffice complet : CRUD activités avec upload d'images WebP redimensionnées par Sharp, CRUD sessions avec blocage si des commandes existent — pour ne pas supprimer une session déjà vendue — et gestion des commandes avec les transitions de statut autorisées.

Les contraintes techniques sont non-négociables pour moi : JWT exclusivement en httpOnly — jamais localStorage — validation Zod sur tous les inputs, argon2id pour les mots de passe, et 151 tests automatisés avec CI verte obligatoire avant tout merge."

SLIDE 04 — Gestion de projet SCRUM

Ce que tu dis :

"5 sprints d'une semaine. Je m'en suis tenu à la Definition of Done : lint propre, tests passants, PR mergée, CI verte. GitHub Projects en Kanban, issues liées aux branches et commits.

Exemple concret de traçabilité : PR #22, suppression des colonnes deleted_at sur activities, categories et sessions — 5 commits atomiques, une migration versionnée, les tests mis à jour, fermée avec 'closes #22'. C'est le niveau de rigueur que j'ai appliqué sur les 24 PR du projet."

SLIDE 05 — Architecture logicielle multicouche

Ce que tu dis :

"Architecture en 4 couches. Le frontend React n'a jamais accès direct à la BDD — tout passe par l'API REST. La couche métier c'est Express 5 avec une chaîne route → middleware Zod → controller → Prisma. La sécurité est transversale : JWT httpOnly pour l'auth, argon2id pour les mots de passe, CORS strict, SameSite=Strict.

Ce qui est important ici, c'est que chaque couche a une responsabilité unique. Zod valide avant d'entrer dans le controller. Prisma isole la BDD du code métier. Le frontend ne sait pas si c'est PostgreSQL ou autre chose derrière."

SLIDE 06 — Modèle de données (MCD)

Ce que tu dis :

"9 entités. Les choix notables : soft delete uniquement sur users et orders — pour la conformité RGPD et la conservation pour audit. Hard delete sur activities, sessions, catégories — intentionnel, pour libérer les fichiers images associés.

Le prix unitaire HT et le taux de TVA sont figés dans orders_lines au moment de la commande. Si un admin modifie le prix d'une activité plus tard, les commandes passées restent intactes — c'est une exigence métier fondamentale.

Pour les refresh tokens : token_id UUID en clair en base pour le lookup O(1). Le JWT entier est hashé via argon2 — token_hash = argon2(JWT). Au refresh, on retrouve la ligne par token_id, puis argon2.verify(cookie, token_hash) confirme que le cookie est authentique. Jamais le JWT brut en base."

SLIDE 07 — Stack technique justifiée

Ce que tu dis :

"Chaque choix est motivé. Express 5 pour la gestion native des erreurs async sans try/catch partout. TypeScript strict zéro-any pour détecter les bugs à la compilation. Prisma pour les requêtes paramétrées — aucune interpolation SQL possible via l'ORM.

argon2id plutôt que bcrypt : c'est la recommandation OWASP 2023, résistant GPU et ASIC. Biome à la place d'ESLint+Prettier — même résultat, 10x plus rapide, une seule config.

Vitest + Supertest : Supertest monte Express directement en mémoire sans port réseau. Les tests sont rapides et sans effet de bord."

SLIDE 08 — Authentification JWT

Ce que tu dis :

"Le flux est simple : login → argon2.verify → génération des deux tokens → Set-Cookie httpOnly. Aucun token ne touche le JavaScript côté client.

Côté frontend, l'intercepteur apiFetch gère le refresh de manière transparente : si une requête reçoit un 401, on déclenche un POST /auth/refresh, puis on re-joue la requête initiale. L'UI ne voit jamais l'expiration.

Le point délicat : React StrictMode double-invoke les effets en développement. Deux appels simultanés à /refresh consommaient le même token — le second échouait avec 401. Solution : une Promise singleton partagée dans apiFetch. Si un refresh est déjà en cours, on attend sa résolution au lieu d'en déclencher un second."

SLIDE 09 — Code & Sécurité

Ce que tu dis :

"Je couvre 5 des 10 catégories OWASP. Les plus importantes : A01 Broken Access Control — le user_id est toujours extrait du JWT, jamais du body. Un utilisateur ne peut pas créer une commande au nom d'un autre. A03 Injection — Prisma paramétré + Zod sur tous les inputs. A07 Auth Failures — rotation systématique du refresh token, révocation en base à chaque logout.

Côté accès données : les transactions Prisma pour les opérations critiques — la vérification de capacité disponible et la création de ligne de commande sont atomiques. Si deux utilisateurs réservent la dernière place simultanément, un seul passe."

SLIDE 10 — Plan de tests — 151 tests

Ce que tu dis :

"Pyramide de tests : priorité aux tests d'intégration — 116 tests — parce qu'ils couvrent la vraie chaîne route → controller → BDD avec le moins de mocking possible. 35 tests unitaires sur les helpers isolés : génération de tokens, slugification, pagination.

La base de test est isolée — `zombiezone_test` — et remise à zéro avant chaque suite. Les tests d'auth utilisent `request.agent` de Supertest pour maintenir les cookies entre requêtes, comme un vrai navigateur le ferait."

SLIDE 11 — Jeu d'essai — Cycle JWT

Ce que tu dis :

"5 scénarios, aucun écart. S1 : login valide, deux cookies `httpOnly` définis. S2 : mauvais mot de passe — même message 'Invalid credentials' qu'un email inconnu, protection anti-oracle. S3 : token expiré, le refresh automatique fonctionne, la rotation est vérifiée en base. S4 : accès sans token, le guard clause du middleware déclenche un 401 immédiat. S5 : logout suivi d'une tentative de refresh avec l'ancien token — refusé, `token_id` absent en base.

Point d'amélioration identifié : absence de rate limiting sur `/login`. J'ai documenté ça dans le backlog comme amélioration post-MVP."

SLIDE 12 — Pipeline CI/CD

Ce que tu dis :

"Le pipeline se déclenche sur chaque push et chaque PR. La stratégie fail-fast : si le lint échoue, les tests ne tournent pas. Si les tests échouent, le build ne se lance pas. Le déploiement SSH sur le VPS n'est déclenché que sur push master.

Le service PostgreSQL de test est éphémère dans le runner GitHub — recréé à chaque run, migrations appliquées. Ça garantit que les tests ne dépendent jamais d'un état laissé par un run précédent."

SLIDE 13 — Déploiement Docker & VPS

Ce que tu dis :

"Docker multi-stage : le stage builder compile TypeScript vers dist/, le stage runner ne contient que le code compilé et `node_modules` — sans les sources TypeScript ni les devDependencies. Image prod allégée.

Sur le VPS Ionos : Nginx termine le SSL, redirige HTTP vers HTTPS, proxifie `/api/` vers le backend sur le réseau Docker interne. Le port PostgreSQL 5432 n'est pas exposé publiquement — accessible uniquement depuis les containers Docker.

Un piège résolu : Prisma cherche les migrations dans `prisma/migrations` relatif au CWD. Notre schéma est dans `backend/src/models/migrations`. Solution : un symlink créé dans le Dockerfile. Documenté dans `DEPLOY.md`."

SLIDE 14 — Difficultés techniques

Ce que tu dis :

"Quatre difficultés notables. La race condition sur le refresh : résolue par la Promise singleton. Le bfcache : un utilisateur qui se déconnecte et appuie sur 'Précédent' pouvait voir son espace client restauré depuis la mémoire du navigateur sans aucun JS

exécuté — résolu par Cache-Control no-store côté serveur et une vérification réseau au montage côté client.

Les migrations Prisma en multi-stage : le symlink dans le Dockerfile. Et Biome en monorepo : l'exécution depuis la racine crée un conflit de configuration, il faut exécuter depuis chaque workspace."

SLIDE 15 — Améliorations futures

Ce que tu dis :

"Deux priorités hautes : le rate limiting sur les routes d'auth — c'est une lacune identifiée pendant les tests — et les emails transactionnels — confirmation de commande avec détail des lignes. L'infrastructure SMTP est déjà en place avec Nodemailer sur Ionos, le flow reset password utilise le même canal.

Stripe en V2 : l'architecture l'anticipe — les commandes ont déjà un champ payment_method. Playwright pour les tests E2E : le flow login → panier → commande → annulation est le candidat naturel."

SLIDES 16-17 — Compétences CDA

Ce que tu dis :

"Les 11 compétences sont couvertes. AT1 : l'environnement monorepo avec CI/CD, les interfaces React, les composants Express, la gestion SCRUM. AT2 : le cahier des charges et les maquettes Figma, l'architecture multicouche documentée, le schéma Prisma avec migrations versionnées, les transactions et le soft delete. AT3 : les 151 tests, la documentation DEPLOY.md, le pipeline GitHub Actions."

SLIDE 18 — Conclusion

Ce que tu dis :

"zØmbie zØne c'est 11 compétences couvertes, 151 tests automatisés, 26 endpoints, déployé en production sur sharo.fr. Stack moderne, sécurité appliquée, documentation à jour.

Je suis prêt pour vos questions."

Ce qui est vérifiable et vrai dans le projet

- 244 commits conventionnels sur GitHub
- 24 PR mergées
- 151 tests (116 intégration + 35 unitaires)
- CI verte
- Application déployée en prod : sharo.fr
- Code public : github.com/5h4r0/z0mbiez0ne
- Headers de sécurité HTTP actifs en prod
- "zéro any TypeScript" → pas absolu

Ce qui n'a pas eu lieu

- GitHub Projects Kanban → pas utilisé
 - Lazy loading images → pas implémenté
 - Sprints formalisés / daily stand-up → pas fait
-

ENTRETIEN TECHNIQUE — Questions probables & Réponses

AUTH & SÉCURITÉ

Q : Pourquoi httpOnly et pas localStorage pour les tokens ?

localStorage est accessible par JavaScript — une faille XSS suffit pour voler le token. Un cookie httpOnly est inaccessible à document.cookie, même en cas de XSS. SameSite=Strict protège contre le CSRF : le cookie n'est pas envoyé depuis un domaine tiers.

Q : Qu'est-ce que la rotation du refresh token ?

À chaque appel /refresh, on supprime l'ancien refresh token en base et on en génère un nouveau. Si un attaquant vole un refresh token et tente de l'utiliser après que le client légitime l'a déjà rotaté, le token_id est absent en base → 401 immédiat.

Q : Que stockes-tu en base pour les refresh tokens, et pourquoi ?

Deux colonnes : token_id (UUID en clair, pour le lookup O(1)) et token_hash (argon2 du JWT entier). Au refresh, on retrouve la ligne par token_id, puis argon2.verify(cookie, token_hash) confirme que le cookie est authentique. Si la base fuit, le JWT hashé est inutilisable sans le secret JWT. Hacher le token_id n'aurait aucun sens — il est déjà en clair dans la même ligne.

Q : Pourquoi argon2id plutôt que bcrypt ?

argon2id est résistant aux attaques GPU et ASIC — il paramètre la mémoire requise, pas seulement le temps de calcul. C'est la recommandation OWASP depuis 2023, supérieur à bcrypt sur ce critère.

Q : Comment fonctionne le guard bfcache ?

Le navigateur peut restaurer une page depuis la mémoire (back/forward cache) sans ré-exécuter le JavaScript. Un utilisateur déconnecté pouvait voir son dashboard. Double protection : Cache-Control no-store sur les routes protégées côté serveur + au montage de chaque page protégée, on appelle refreshToken() sur le réseau, pas seulement l'état Zustand en mémoire.

Q : Comment tu gères le user_id dans les commandes ?

user_id est toujours extrait de req.user.id — le payload du JWT vérifié par le middleware requireAuth. Jamais du body. C'est la correction BUG-B1 : sans ça, un utilisateur pouvait envoyer n'importe quel user_id dans le body et créer des commandes au nom d'autrui.

BASE DE DONNÉES & PRISMA

Q : Pourquoi soft delete sur users/orders et hard delete sur activities/sessions ?

Users et orders : obligation légale de traçabilité RGPD et audit des achats. Hard delete supprimerait l'historique des commandes d'un utilisateur supprimé.

Activities/sessions : suppression physique intentionnelle pour libérer les fichiers images associés sur le disque. Un soft delete laisserait des images orphelines.

Q : Pourquoi figer le prix dans orders_lines ?

Si un admin modifie le prix d'une activité à 50€ alors qu'elle était à 30€ au moment de l'achat, la commande historique doit rester à 30€. unit_price_ht et vat_rate sont copiés à la création de la ligne, indépendants des évolutions tarifaires.

Q : Comment fonctionne la transaction createOrder ?

La vérification de capacité disponible et la création de la ligne de commande sont dans la même transaction Prisma. Ça évite le problème de deux utilisateurs qui réservent simultanément la dernière place — la transaction est atomique, le second commit lèvera une erreur si la capacité est dépassée.

Q : Pourquoi normalisation 3NF ?

Chaque attribut dépend uniquement de la clé primaire, pas d'autres attributs non-clé. Ça évite les anomalies de mise à jour : si le nom d'une catégorie change, on le change à un seul endroit, pas dans toutes les activities_categories.

ARCHITECTURE & CHOIX TECHNIQUES

Q : Pourquoi Prisma plutôt qu'un ORM plus classique comme Sequelize ?

Prisma génère un client entièrement typé à partir du schéma. Le `select` explicite est idiomatique — on déclare exactement les colonnes retournées, ce qui empêche les fuites de données comme le `password_hash`. Les migrations sont versionnées et reproductibles.

Q : Pourquoi React Router 7 et non v6 ?

React Router 7 est l'unification de `remix` et `react-router`. Plus besoin du package `react-router-dom` séparé. Les loaders permettent de charger les données avant le rendu du composant — meilleure UX, pas de flash de contenu vide.

Q : Pourquoi Zustand plutôt que Redux ?

Zustand est minimaliste — pas de boilerplate actions/reducers. Le store est un hook React standard. Pour ce projet, l'état global se limite au user authentifié et au panier — Redux serait surdimensionné.

Q : Pourquoi Docker multi-stage ?

Le stage `builder` contient Node + TypeScript + `devDependencies` — lourd. Le stage `runner` ne contient que le `dist/` compilé et les `node_modules` de production. Image finale 3x plus légère, surface d'attaque réduite.

TESTS

Q : Pourquoi prioriser les tests d'intégration sur les tests unitaires ?

Pour un backend Express, les tests d'intégration couvrent la vraie chaîne — routing, middleware, controller, Prisma, BDD. Un test unitaire sur le controller seul mocke trop et ne teste pas les interactions réelles. Le ROI est meilleur.

Q : Comment tu isolas les tests ?

Base de test dédiée `zombiezone_test` avec `TEST_DATABASE_URL`. La fonction `resetDatabase()` vide toutes les tables dans l'ordre des FK avant chaque suite. Aucun test ne dépend de l'état laissé par le test précédent.

Q : `request.agent` — pourquoi ?

Supertest sans agent ne maintient pas les cookies entre requêtes. Pour les tests d'auth — login puis accès à une route protégée — il faut que le cookie `Set-Cookie` du login soit automatiquement envoyé sur la requête suivante. `request.agent(app)` gère ça.

DÉPLOIEMENT & CI/CD

Q : Comment fonctionne le pipeline en cas d'échec ?

Fail-fast : chaque étape est conditionnée à la réussite de la précédente. Lint échoue → les tests ne tournent pas. Tests échouent → pas de build. Pas de build → pas de déploiement. Sur une PR, le merge est bloqué si la CI est rouge.

Q : Comment tu gères les secrets en production ?

Les secrets GitHub (SSH_KEY, DB_URL, JWT secrets) sont injectés comme variables d'environnement dans le runner CI. Sur le VPS, ils sont dans `backend/.env.production` — jamais committé dans le repo. Le `.gitignore` exclut tous les fichiers `.env`.

Q : Pourquoi Nginx devant les containers Docker ?

Nginx termine le SSL (certificat Let's Encrypt) et redirige HTTP vers HTTPS. Il proxifie `/api/` vers le backend sur le réseau Docker interne. Le frontend React est servi comme fichiers statiques par Nginx — pas de Node.js pour le HTML/JS/CSS.

QUESTIONS OUVERTES PROBABLES

Q : Qu'est-ce que tu ferais différemment si tu recommençais ?

Je mettrais en place le `rate limiting` dès le sprint 2 — c'est une lacune identifiée. J'anticiperais aussi mieux la gestion des uploads en Docker : il faut un volume partagé entre backend et nginx pour servir les images uploadées, ce que j'ai résolu mais

tardivement.

Q : Comment tu améliorerais la sécurité ?

Rate limiting express-rate-limit sur /auth/. Helmet.js pour les headers de sécurité HTTP (CSP, HSTS, X-Frame-Options).
Monitoring des tentatives de login échouées. Rotation des secrets JWT périodiquement.*

Q : Le RGPD dans ton projet ?

*Soft delete sur users et orders — les données sont masquées mais conservées pour l'audit. Klaro pour le consentement cookies.
Politique de cookies documentée. Le droit à l'effacement est implémenté — l'utilisateur peut supprimer son compte depuis son profil (deleted_at = now()).*

QUESTIONNAIRE PROFESSIONNEL (anglais)

Format : lecture doc technique en anglais, 2 QCM en français, 2 questions ouvertes en anglais.

Ce que tu peux revoir :

- Vocabulaire tech en anglais : middleware, payload, token rotation, soft delete, race condition, atomic transaction, reverse proxy, ephemeral service
- Formuler des réponses courtes : "The purpose of X is to Y", "This approach ensures Z"

Exemples de questions courtes en anglais :

- "What is the purpose of the httpOnly flag on a cookie?" → "It prevents JavaScript from accessing the cookie, protecting against XSS attacks."
- "Why is token rotation used with refresh tokens?" → "Each refresh generates a new token and invalidates the old one, so a stolen token becomes useless after the legitimate client has used it."

ENTRETIEN FINAL (20 min — dossier professionnel)

Sur le DP :

- Projet solo = tous les rôles endossés → montrer la polyvalence
- 24 PR, 244 commits → discipline et traçabilité
- Formation O'Clock + projets autodidactes en amont → parcours de reconversion motivé

Question classique : "Qu'est-ce que ce projet vous a appris ?"

"La gestion de la complexité distribuée — faire travailler ensemble Docker, Nginx, Node, PostgreSQL en prod m'a appris que la documentation (DEPLOY.md) est aussi importante que le code. Et la sécurité dès la conception : intégrer httpOnly, argon2id et Zod dès le sprint 2 m'a évité des refactorisations coûteuses plus tard."